

GyaanSetu Frontend Deep Dive

A Comprehensive, Code-by-Code Technical Analysis of the React 18, Vite & Tailwind CSS Architecture for Placement Preparation

Author / Candidate:	Placement Candidate (Aditya)
Project Name:	GyaanSetu – Mentor-Student Doubt Solving Platform
Frontend Stack:	React 18, Vite, React Router v6, Axios, Tailwind CSS, Lucide React Icons
Document Scope:	Complete source component analysis, React Hooks breakdown, AuthContext & routing logic
Target Role:	Frontend Developer / React Engineer / Full-Stack SDE

Table of Contents

1. Frontend Overview & Architecture

2. Actual Frontend Folder Structure

3. Core Files (main.jsx, App.jsx, index.css)

4. Client Routing & Protected Routes

5. Auth Context & State Management

6. Central Axios Service & Interceptors

7. Reusable Components Breakdown
8. Pages Detailed Analysis

9. React Hooks Deep Dive

10. Conditional UI & Dual-Role Rendering

11. End-to-End Frontend User Flows

12. Controlled Forms & API States

13. Tailwind CSS & UI Design Patterns

14. Frontend Placement Revision Notes

1. Frontend Overview & Architecture

The GyaanSetu frontend is a Single Page Application (SPA) built using React 18, Vite, and Tailwind CSS. It communicates asynchronously with the Express backend API via Axios. Client-side routing is handled by React Router v6, and user identity state is managed globally using React Context (`AuthContext`).

Why this frontend stack was chosen:

- **React 18:** Component-driven architecture allowing fast re-renders, virtual DOM diffing, and declarative UI states.
- **Vite:** Next-generation build tool providing instantaneous Hot Module Replacement (HMR) and optimized Rollup production bundling.
- **React Router v6:** Provides client-side dynamic route navigation without page reloads.
- **Axios:** Promise-based HTTP client equipped with request interceptors for automated Bearer token injection.
- **Tailwind CSS:** Utility-first CSS framework delivering clean responsive styling without heavy CSS bloat.

Frontend Component Interaction Lifecycle:

User Action (e.g. Click Button) -> Event Handler (handleSubmit) -> API Call (Axios Service) -> Backend REST API -> JSON Response -> State Update (useState / AuthContext) -> React UI Re-render

2. Actual Frontend Folder Structure

The frontend code lives in `client/` and is organized modularly by concerns:

```

client/
├── src/
│   ├── components/
│   │   ├── Navbar.jsx           # Top navigation bar with role badges & persona actions
│   │   ├── BecomeMentorModal.jsx # Modal form to submit technical skills and become mentor
│   │   ├── RequestDoubtModal.jsx # Modal form to send doubt requests to a target mentor
│   │   ├── ScheduleModal.jsx     # Modal form for mentors to pick date & time for session
│   │   ├── ProtectedRoute.jsx    # Route guard component restricting unauthenticated users
│   │   └── context/
│   │       └── AuthContext.jsx    # React Context holding user state, JWT token, login & logout
│   ├── pages/
│   │   ├── Home.jsx             # Landing page with CTA hero & feature cards
│   │   ├── Login.jsx            # Login page with clean single account login form
│   │   ├── Register.jsx         # User registration page
│   │   ├── Mentors.jsx          # Mentor directory with real-time name & skill search
│   │   └── Dashboard.jsx        # Unified dashboard showing student requests & mentor request
│   ├── sessions/
│   │   ├── Sessions.jsx         # Mentoring sessions list with Jitsi join links
│   │   └── Profile.jsx          # User profile & skill editor page
│   ├── services/
│   │   └── api.js               # Axios client instance with Authorization Bearer header interceptor
│   ├── App.jsx                 # Main router setup & global modal container
│   ├── main.jsx                 # React DOM root entry point
│   └── index.css                # Tailwind CSS directives & global base styles
├── package.json                # Frontend dependencies (React, Vite, React Router, Axios, Lucide)
└── vite.config.js

```

```

■■■ vite.config.js           # Vite dev server & API proxy configuration
■■■ tailwind.config.js       # Tailwind theme customization
■■■ postcss.config.js        # PostCSS Tailwind processor config
■■■ vercel.json              # Vercel deployment rewrite rule for SPA routing

```

3. Core Files (main.jsx, App.jsx, index.css)

A. main.jsx: Initializes the React DOM root element and renders `` inside ``.

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);

```

B. App.jsx: Encloses the application in `` and `` , defines page routes, and holds state for the global ``.

```

import React, { useState } from 'react';
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import Navbar from './components/Navbar';
import BecomeMentorModal from './components/BecomeMentorModal';
import ProtectedRoute from './components/ProtectedRoute';
// Pages import...

function App() {
  const [isBecomeMentorModalOpen, setIsBecomeMentorModalOpen] = useState(false);
  return (
    <AuthProvider>
      <Router>
        <Navbar onOpenBecomeMentor={() => setIsBecomeMentorModalOpen(true)} />
        <Routes>
          <Route path="/" element={<Home onOpenBecomeMentor={() => setIsBecomeMentorModalOpen(true)} />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="/mentors" element={<Mentors />} />
          <Route path="/dashboard" element={<ProtectedRoute><Dashboard onOpenBecomeMentor={() => setIsBecomeMentorModalOpen(true)} /></ProtectedRoute>} />
          <Route path="/sessions" element={<ProtectedRoute><Sessions /></ProtectedRoute>} />
          <Route path="/profile" element={<ProtectedRoute><Profile onOpenBecomeMentor={() => setIsBecomeMentorModalOpen(true)} /></ProtectedRoute>} />
        </Routes>
        <BecomeMentorModal isOpen={isBecomeMentorModalOpen} onClose={() => setIsBecomeMentorModalOpen(false)} />
      </Router>
    </AuthProvider>
  );
}

```

4. Client Routing & Protected Routes

Routing is configured using React Router v6. Protected pages (`/dashboard`, `/sessions`, `/profile`) are wrapped in ``.

Path	Component Rendered	Access Level	Purpose
`/`	`Home.jsx`	Public	Landing hero page & platform overview
`/login`	`Login.jsx`	Public	User authentication login form
`/register`	`Register.jsx`	Public	User account registration form
`/mentors`	`Mentors.jsx`	Public	Mentor directory with skill search & doubt request trigger
`/dashboard`	`Dashboard.jsx`	Protected	Unified dashboard for student requests & mentor incoming requests
`/sessions`	`Sessions.jsx`	Protected	List of scheduled & completed mentoring sessions with Jitsi URLs
`/profile`	`Profile.jsx`	Protected	User details view & mentor skills editor

5. Auth Context & State Management (AuthContext.jsx)

`AuthContext` provides global user identity state across all components without prop drilling.

```
import React, { createContext, useState, useEffect } from 'react';
import API from '../services/api';

export const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(localStorage.getItem('gyaansetu_token') || null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const checkLoggedInUser = async () => {
      if (token) {
        try {
          const response = await API.get('/auth/me');
          if (response.data.success) setUser(response.data.user);
        } catch (error) { logout(); }
      }
      setLoading(false);
    };
    checkLoggedInUser();
  }, [token]);

  const login = (userData, userToken) => {
    localStorage.setItem('gyaansetu_token', userToken);
    setToken(userToken);
    setUser(userData);
  };

  const logout = () => {
    localStorage.removeItem('gyaansetu_token');
    setToken(null);
  };
}
```

```

    setUser(null);
  };
  const updateUser = (updatedUser) => setUser(updatedUser);
  return (
    <AuthContext.Provider value={{ user, token, loading, login, logout, updateUser }}>
      {children}
    </AuthContext.Provider>
  );
};

```

6. Central Axios Service & Interceptors (api.js)

`client/src/services/api.js` creates a single Axios client configured with a request interceptor.

```

import axios from 'axios';

const API = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api'
});

// Automatically attach Authorization Bearer token to all outgoing protected requests
API.interceptors.request.use((config) => {
  const token = localStorage.getItem('gyaansetu_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
}, (error) => Promise.reject(error));

export default API;

```

Why Interceptors are crucial: Prevents manual token handling on every API call. Every request automatically includes `Authorization: Bearer`.

7. Reusable Components Breakdown

A. Navbar.jsx: Renders logo branding, main menu links (`Home`, `Mentors`, `Dashboard`, `Sessions`), and user persona status. If `user.isMentor === false`, displays the 'Become a Mentor' CTA button.

B. BecomeMentorModal.jsx: Modal form allowing logged-in students to enter comma-separated skills and call `PATCH /api/users/become-mentor`.

C. RequestDoubtModal.jsx: Modal form triggered from mentor cards allowing students to submit technical doubts via `POST /api/proposals`.

D. ScheduleModal.jsx: Modal dialog used by mentors to pick date & time (`POST /api/sessions`), checking conflicts and generating Jitsi URLs.

E. ProtectedRoute.jsx: Checks `token` from `AuthContext`. If unauthenticated, redirects to `/login` using ``.

8. Pages Detailed Analysis

- **Home.jsx:** Hero landing page showcasing tagline 'Connect. Learn. Solve.', CTA buttons, and platform overview cards.
- **Login.jsx & Register.jsx:** Controlled forms managing auth submission, error alerts, and redirect handling.
- **Mentors.jsx:** Renders mentor directory with client-side real-time filtering by name and skills. Prevents self-requesting by hiding the 'Ask Doubt' button on the user's own card.
- **Dashboard.jsx:** Unified dual-role dashboard. Renders 'My Requests' for student mode and 'Incoming Requests' for mentor mode.
- **Sessions.jsx:** Displays scheduled & completed sessions with direct external Jitsi video call links and 'Mark Complete' action buttons.
- **Profile.jsx:** User details overview and skill tag editor for mentors.

9. React Hooks Deep Dive

React Hook	Purpose & Usage in GyaanSetu	Example Usage File
<code>`useState`</code>	Local component state management (form inputs, loading spinners, modal visibility)	<code>`Login.jsx`</code> , <code>`Mentors.jsx`</code>
<code>`useEffect`</code>	Side-effect execution on mount (fetching mentors, sync auth token, fetching dashboard data)	<code>`Dashboard.jsx`</code> , <code>`Mentors.jsx`</code>
<code>`useContext`</code>	Consuming global <code>`AuthContext`</code> state (<code>`user`</code> , <code>`login`</code> , <code>`logout`</code>) without prop drilling	<code>`Navbar.jsx`</code> , <code>`Dashboard.jsx`</code>
<code>`useNavigate`</code>	Programmatic client-side navigation between routes	<code>`Register.jsx`</code> , <code>`Login.jsx`</code>
<code>`useLocation`</code>	Accessing current URL location & state passed during navigation	<code>`Login.jsx`</code>

10. Conditional UI & Dual-Role Rendering

The UI dynamically renders capabilities based on ``user.isMentor``:

- If ``isMentor === false``: Shows 'Become a Mentor' amber CTA button.
- If ``isMentor === true``: Shows 'Active Mentor' badge and enables 'Incoming Requests' and 'My Mentoring Sessions' on Dashboard.

11. End-to-End Frontend User Flows

Complete Lifecycle Flow:

1. User registers (``/register``) → redirects to Login (``/login``).
2. Logged-in student browses Mentors (``/mentors``), searches 'React', clicks 'Ask Doubt'.
3. Mentor logs in, opens Dashboard (``/dashboard``), sees incoming request, clicks 'Accept' then 'Schedule Session'.
4. Both users open Sessions (``/sessions``) and click 'Join Meeting' (opens Jitsi URL in new tab).
5. Student clicks 'Become a Mentor' in top bar, enters skills → account gains full mentor capabilities!

12. Controlled Forms & API States

All inputs use controlled components (``value={email} onChange={(e) => setEmail(e.target.value)}``). Submit buttons handle loading disable state (``disabled={loading}``) to prevent double submissions.

13. Tailwind CSS & UI Design Patterns

Uses a clean, placement-ready blue/indigo design system (``bg-blue-600``, ``hover:bg-blue-700``, ``bg-amber-500``, ``rounded-xl``, ``shadow-sm``) with Lucide React icons (``GraduationCap``, ``Users``, ``Calendar``, ``Video``).

14. Frontend Placement Revision Notes

Key Frontend Concepts to Speak in Interviews:

- **Single Page Application (SPA):** Only one HTML page (``index.html``) is loaded. React Router updates the view dynamically without browser page reloads.
- **Virtual DOM:** React keeps a lightweight in-memory copy of the DOM. When state changes, React computes diffs and updates only changed real DOM nodes.
- **Controlled Components:** Form input state is bound to React state (``useState``), making React the single source of truth.
- **Axios Interceptors:** Centralizes auth token headers, so individual components do not need to manually pass Authorization headers.
- **Context API:** Eliminates prop drilling by making auth state globally accessible via ``useContext(AuthContext)``.